VÉRIFICATION ON-THE-FLY DE PROPRIÉTÉS CTL SUR DES STRUCTURES DE KRIPKE ACYCLIQUES

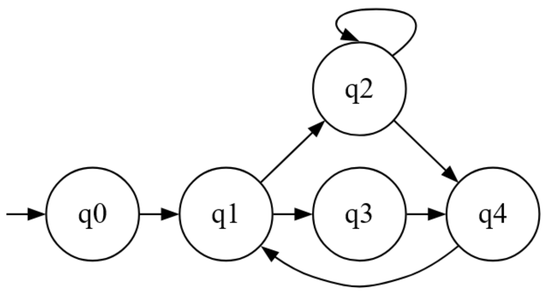
COTTIGNIES Mattew, JENY JEYARAJ John-Michael, EREHAIHI Sayfe-Din
Encadrant : Johan Arcile

Model Checking

Model Checking est une méthode automatique pour vérifier qu'un système est fiable. Elle explore toutes les situations possibles pour prouver mathématiquement que le modèle respecte bien les règles prévues.

Structure de Kripke

Les systèmes sont modélisés par des structures de Kripke $M = (Q, q_0, T, AP, L)$, où Q est l'ensemble des états, q_0 l'état initial, T la relation de transition, AP les propositions atomiques et L la fonction d'étiquetage.

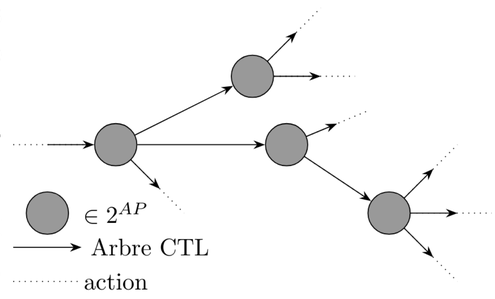


Computation Tree Logic

Les formules CTL se construisent avec les opérateurs logiques et l'association obligatoire de deux opérateurs temporels :

- **Chemins** : **A** : Sur *tous* les chemins, **E** : Sur *au moins un* chemin.
- **Temps** : **X** : À l'état suivant direct, **F** : Finit par devenir vrai, **G** : Reste vrai tout le temps.

Exemple : **EF p** = Sur *au moins un* chemin (E), p *finit par devenir vrai* (F).

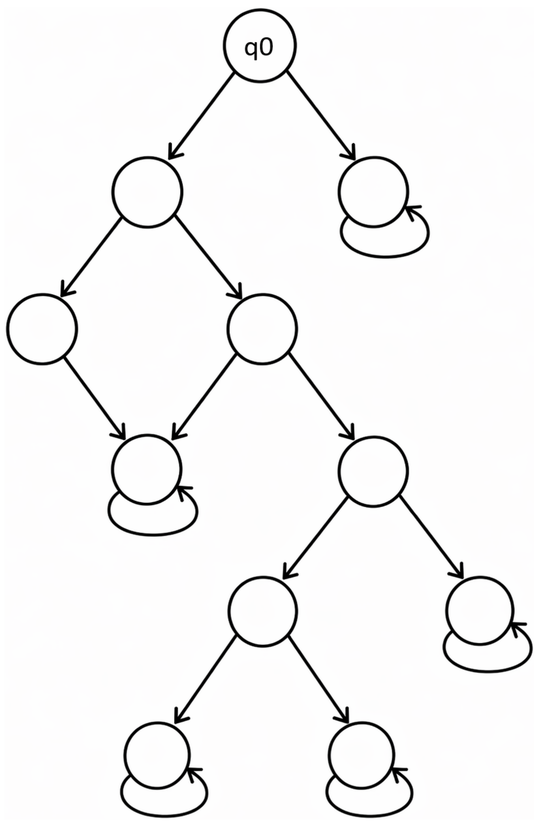


Objectif :

Développer un algorithme de model checking de "Vérification à la volée" pour vérifier des propriétés CTL sur des structures de Kripke acycliques, en évitant l'exploration complète de l'espace d'états.

Sous-classe de modèle

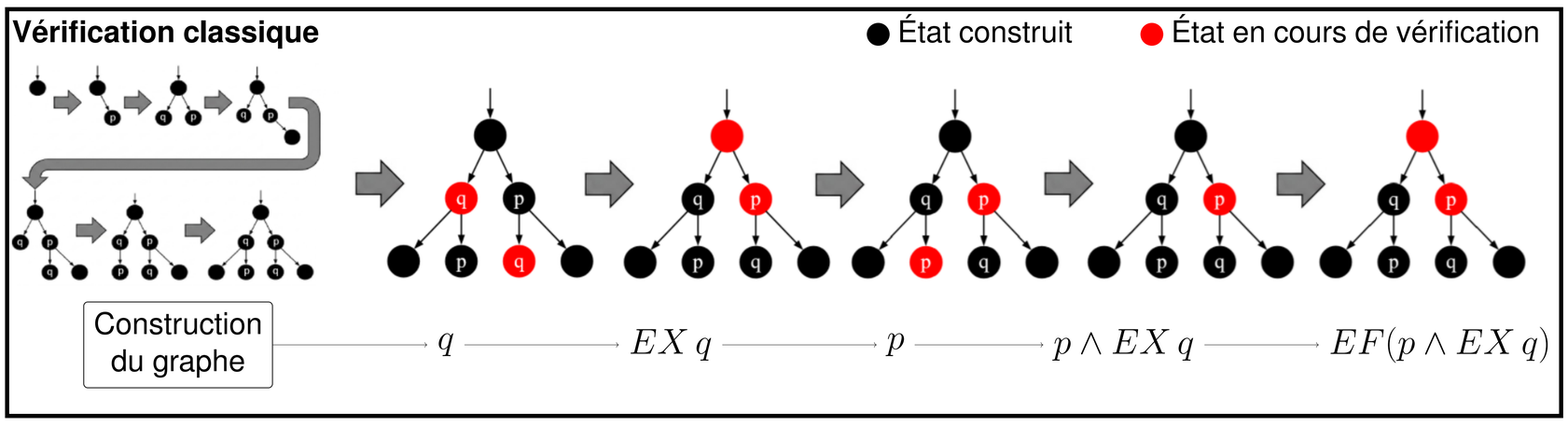
- Modèle de Kripke acyclique (DAG) avec états puits sans successeur.
- Ajout d'une auto-boucle uniquement sur les états puits pour interpréter CTL.



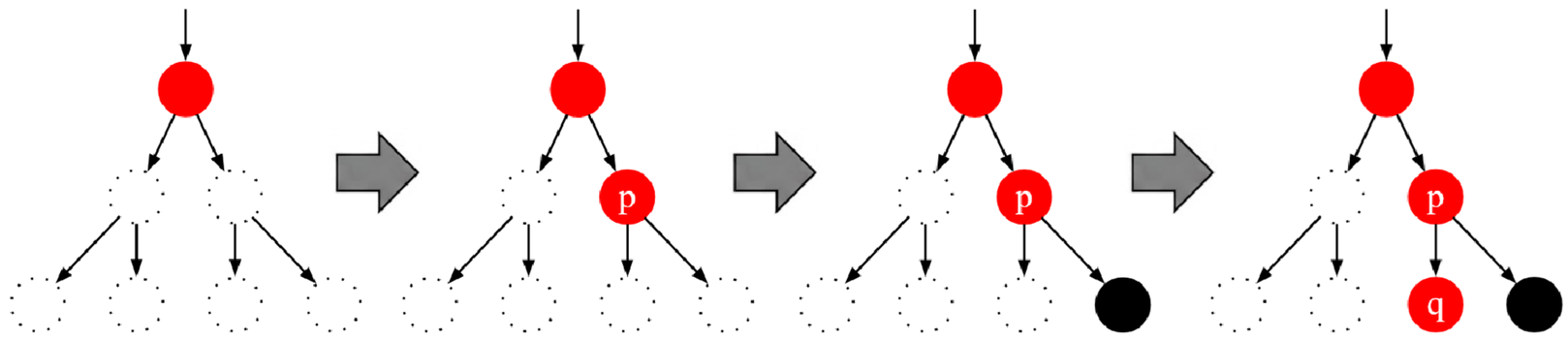
Vérification à la volée

Le graphe est exploré dynamiquement (DFS) sans être entièrement construit. Les figures suivantes détaillent le comportement en vérification basique puis en vérification optimisée pour EF(p ∧ EX q).

Vérification classique



Vérification à la volée



Cette vérification s'arrête dès que la propriété est vérifiée ou réfutée, sans explorer tout le graphe. Il suffit de trouver un état p avec un successeur q, l'algorithme s'arrête dès qu'une réponse est trouvée.

Contribution

Equivalences

Les équivalences suivantes ramènent ces formules à l'atteinte d'une feuille satisfaisant φ :

$$EF(EG(\varphi)) \equiv EF(\varphi \wedge feuille)$$
$$EG(EF(\varphi)) \equiv EF(\varphi \wedge feuille)$$
$$EF(AG(\varphi)) \equiv EF(\varphi \wedge feuille)$$
$$AF(EG(\varphi)) \equiv AF(\varphi \wedge feuille)$$
$$AG(AF(\varphi)) \equiv AF(\varphi \wedge feuille)$$
$$AF(AG(\varphi)) \equiv AF(\varphi \wedge feuille)$$

$EG(AF(\varphi)) =$ Exception
(ne suit pas le pattern $\varphi \wedge feuille$)

Exemples (Equivalences)

Structure du système :

- Le système est un DAG
- Tous les chemins mènent vers des états puits
- Les états puits sont des boucles stables
- Chaque état puits satisfait la propriété p

Interprétation CTL :

- Atteindre une feuille = état stable
- p invariant sur les feuilles
- Toute exécution finit dans une feuille

Conclusion :

$EF(EG(p)) \equiv EG(EF(p)) \equiv EF(p \wedge feuille)$

Algorithme simple à la volée

Vérification de $EF(p \wedge EX\,q)$:

- Exploration du graphe en profondeur depuis l'état initial
- Pour chaque état : vérifier $p(s)$ et $\exists s' \in$ Succ(s) avec $q(s')$
- Si oui : arrêter et valider la propriété
- Sinon : continuer l'exploration
- Arrêt dès qu'une solution est trouvée ou tout est exploré

Check(s):

si $p(s)$ et $\exists s' \in$ Succ(s) avec $q(s')$:
  retourner vrai
pour chaque $s' \in$ Succ(s) :
  si Check$(s') =$ vrai :
    retourner vrai
retourner faux
Résultat $=$ Check(s_0)

Perspectives

- Généraliser les simplifications obtenues dans les structures de Kripke acycliques à des sous-ensembles différents
- Étendre l'algorithme on-the-fly à des fragments plus larges de CTL, notamment les formules fortement imbriquées
- Optimiser l'implémentation sous forme de code afin de mieux visualiser les résultats de généralisation
- Intégrer la vérification dans un outil logiciel complet de Vérification à la volée;